

INFOGRAFÍA TÉCNICA

CICLO DE VIDA DEL SOFTWARE Y COMPARACIÓN DE METODOLOGÍAS (CLÁSICAS, ÁGILES Y WEB3)

CICLO DE VIDA DEL SOFTWARE (SDLC)

Es el marco de trabajo estructurado que define las fases y actividades necesarias para planificar, diseñar, construir, probar, desplegar y mantener un sistema de software con altos estándares de calidad.

FASES DEL DESARROLLO

- **1. Planificación y Análisis de Requisitos:** Se levantan las necesidades del cliente, alcance, restricciones y requerimientos funcionales y no funcionales.
- **2. Diseño del Sistema:** Se define la arquitectura de software, modelado de bases de datos, interfaces de usuario (UI/UX) y componentes de red.
- **3. Desarrollo e Implementación:** Se escribe la lógica del sistema mediante código fuente en el lenguaje de programación elegido.
- **4. Pruebas y Validación (QA):** Se ejecutan pruebas de rendimiento, seguridad, integración y usabilidad para corregir defectos.
- **5. Despliegue (Puesta en Producción):** Se instala el software en el entorno final de producción para que los usuarios puedan utilizarlo.
- **6. Mantenimiento y Evolución:** Se monitorea el sistema para corregir fallas operativas, aplicar parches de seguridad y agregar nuevas funciones.

COMPARACIÓN DE METODOLOGÍAS DE DESARROLLO

1. METODOLOGÍAS CLÁSICAS / TRADICIONALES

Ejemplos: Modelo en Cascada (Waterfall), Modelo en V, RUP.

¿Quién decide?: El Project Manager y los líderes arquitectos según la planificación inicial.

Gestión de Cambios: Rigurosa y compleja; los cambios avanzados generan alto impacto y reproceso.

Costo: Elevado si hay desviaciones o modificaciones en etapas tardías.

Ventaja: Documentación exhaustiva, alta predictibilidad y control estricto de tiempos.

Limitación: Poca flexibilidad para adaptarse a nuevos requerimientos del mercado.

2. METODOLOGÍAS ÁGILES

Ejemplos: Scrum, Kanban, Extreme Programming (XP).

¿Quién decide?: Equipo multidisciplinario de forma colaborativa junto con el Product Owner.

Gestión de Cambios: Alta adaptación; se integran cambios continuamente al finalizar cada iteración o Sprint.

Costo: Dinámico y controlado mediante entregables continuos e incrementales.

Ventaja: Entregas rápidas de valor, flexibilidad constante y alineación con el cliente.

Limitación: Requiere comunicación constante y puede haber menor documentación formal.

3. METODOLOGÍAS WEB3 Y CRIPTO-DESARROLLO

Ejemplos: Desarrollo dApps, Contratos Inteligentes (Solidity, Rust), DAO Governance.

¿Quién decide?: Comunidad descentralizada (gobernanza por tokens) y desarrolladores core.

Gestión de Cambios: Muy compleja; los contratos inteligentes inmutables requieren auditorías estrictas e implementación de hard forks o proxies para actualizarse.

Costo: Elevado por auditorías de seguridad en blockchain y comisiones de red (gas fees).

Ventaja: Inmutabilidad, transparencia, seguridad criptográfica y ausencia de intermediarios.

Limitación: Alta complejidad en pruebas, riesgos irreversibles por fallos en código y baja curva de corrección.